COMP2511

Tutorial 9

Notices

- Assignment-ii Part 2 is due **next Wednesday 3pm**.
  - Remember that Part 2 is to be submitted on **Moodle**, as a single PDF.
- A sample exam environment will be running on the lab computers next week.
  - The aim is to get you comfortable with the **structure**, **layout** and **available materials** of the exam.
  - The difficulty of the sample exam will not necessarily be indicative of the actual exam.

Software Architecture

- Suppose you've built an online banking app that looks great with all features working correctly.
- The application crashes if more than 100 users are using it at the same time.
- Would this be considered a success?
  - Not necessarily; the application isn't going to be very useful if it can't serve many users at once.
  - The system should have the ability to handle growth in user traffic without compromising performance. This is an idea referred to as **scalability**.
  - 'Good' software needs to take factors like this into account.

Architectural Characteristics

- **Architectural characteristics** are *non-functional requirements* that define the qualities or attributes a system should exhibit to meet operational needs.
- Unlike *functional requirements*, they are often not explicitly defined.
  - This adds another layer of complexity to designing software; we have to be aware of these requirements ourselves.

Architectural Characteristics

- Examples of architectural characteristics (not exhaustive):
  - **Scalability**: The ability to handle increased workload.
  - **Performance**: The responsiveness and speed of the system.
  - **Availability**: The uptime of the system.
    - **Fault Tolerance**: The ability to continue operating if/when a component fails.
  - **Reliability**: The system's ability to consistently and correctly do what it's supposed to.
  - **Maintainability**: The ease at which the system can be modified or repaired over time.
  - **Security**: The system's protection against unauthorised access and data breaches.
  - **Testability**: The extent to which the system can be easily and effectively tested.
  - **Team Autonomy**: The ability for separate teams working on different things to work and deploy components independently.

Architectural Characteristics

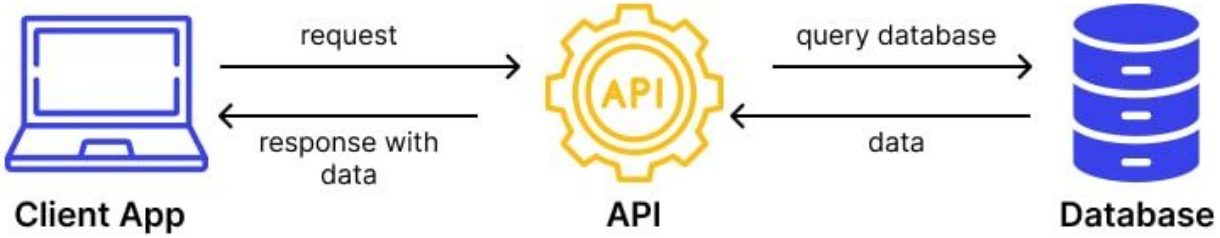
- You will find that there are usually **trade-offs** between characteristics.
  - For example, increasing security → decrease in performance
- Depending on the context, some characteristics will be more important than others.
- When making architectural decisions, your job is identifying which characteristics are the highest priority.

Architectural Styles

- **Architectural styles** are predefined patterns and philosophies guiding how software systems are structured.
- There are two main categories of structuring an application:
  - **Monolithic**: The system is a single application that handles everything.
  - **Distributed**: The system is composed of multiple small, independent applications that communicate with each other.
- Each style comes with its own set of strengths and weaknesses regarding architectural characteristics. A *hybrid* of architectural styles can be used.
- The style of a system will influence how it behaves and evolves. It is very difficult to convert a system from one style to another!

Application Programming Interfaces (APIs)

- An API is a ‘messenger’ that allows different applications to communicate and exchange data with each other.
  - Think of an API as a waiter that accepts an order, sends it to the kitchen, and delivers the order once it’s ready.
  - This is (mostly) how the components of a distributed system interact with each other. These requests take some time to handle (**latency**).

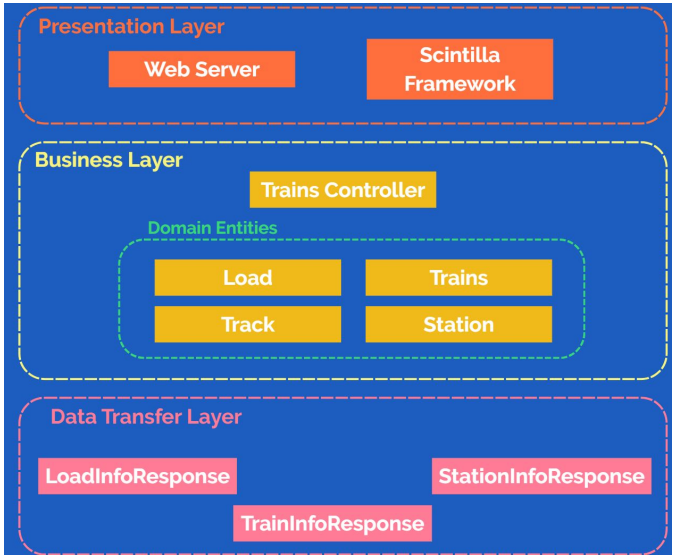


Data Consistency

- This refers to the uniformity of data and information across a system.
- Ideally, all users of an application should access the same, up-to-date information at all times.
  - Changing details on social media
  - Stock counts in stores
- With a monolithic architecture, there is (typically) only a single database used in the entire application so this isn't a concern.
- With a distributed architecture, each component handles its own database, and changes in one take time to be reflected in another.
  - Data in distributed systems is usually **eventually consistent**; changes aren't immediately reflected between databases, but they will eventually line up if no changes occur for a while.
  - **CAP Theorem**: You can only have 2/3 of Consistency, Availability or Partition Tolerance.

Layered Architecture

- A layered architecture is a **monolith** where responsibilities are separated into different technical layers (Presentation, Business, Data), defining a clear separation of concerns.

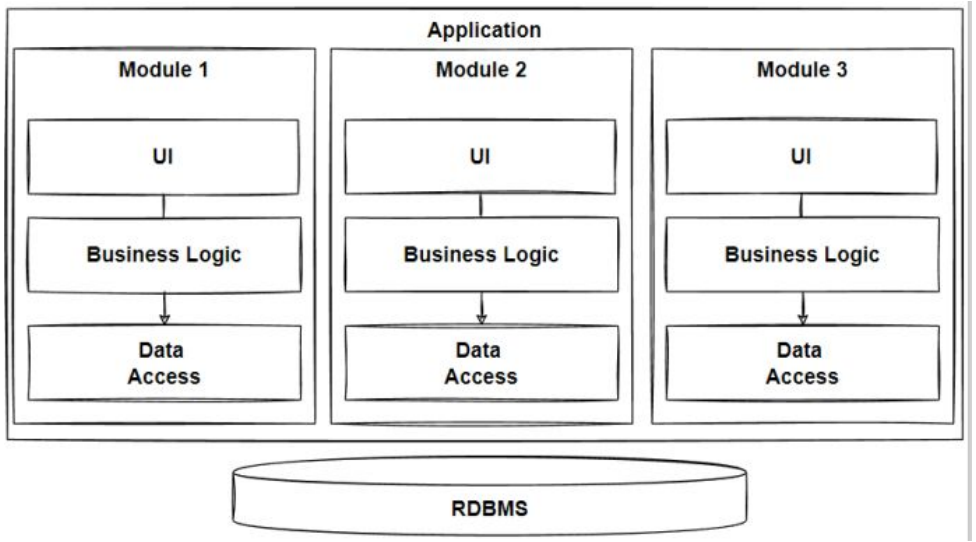


Layered Architecture

- Relation to architectural characteristics:
  - **Simplicity**: in general, this would be the most intuitive approach and is the easiest to 'get up and running'.
  - **Data Consistency**: all layers share a single database, so data is always consistent.
  - **Maintainability**: making changes within layers is simple, but difficult across layers.
  - **Performance**: good for smaller applications since in-process calls are faster than API/network calls, but bad for a large volume of requests.
  - **Security**: developers only need to focus on protecting a single application, but there is a 'single point of attack'; if an attacker gains access to one part, this can compromise the whole system.
  - **Scalability**: individual components cannot be scaled without scaling the rest of the app.
  - **Fault Tolerance**: if one part of the system crashes, the rest of the system is likely to also crash.

Modular Monolithic Architecture

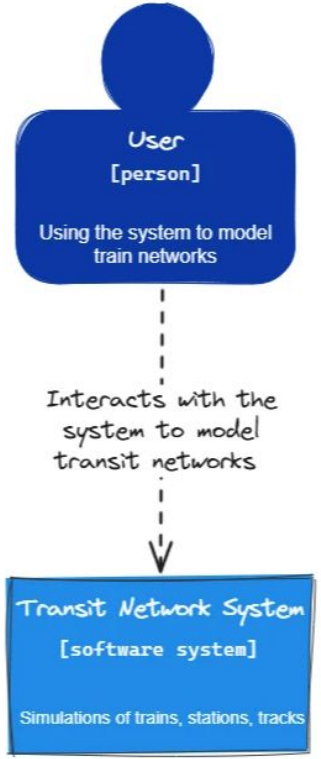
- A modular monolith is a monolith that is organised according to different business domains, rather than technical layers.
- Each module is responsible for a specific functional requirement.

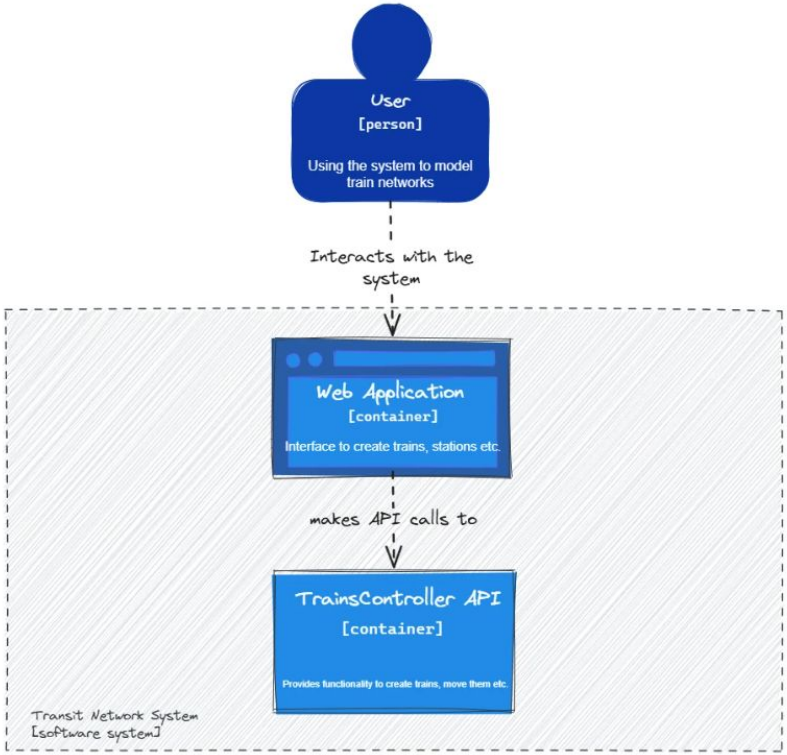


Modular Monolithic Architecture

- Relation to architectural characteristics:
  - **Simplicity**: still simple; not as simple as a layered architecture, but provides some benefits of a microservice architecture while keeping the system as a single application.
  - **Data Consistency**: all modules share a single database, so data is always consistent.
  - **Maintainability**: easier to manage code within the same module; may be tricky across modules.
  - **Security**: each module can have its own mode of authorisation with some modules having more tighter controls than others. Still a ‘single point of attack’, however.
  - **Scalability**: independent modules can be scaled, but there may still be bottlenecks in processing power and database capacity due to the monolithic structure.
  - **Fault Tolerance**: if one part of the system crashes, the rest of the system is likely to also crash.

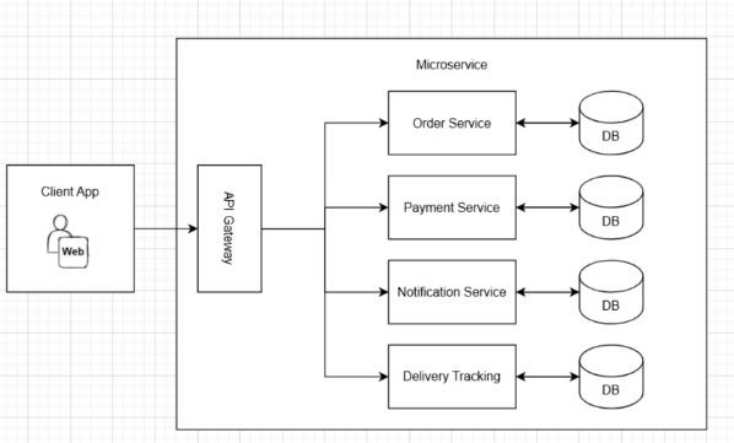
Monoliths in C4 Diagrams (same for Layered and MM)





Microservice Architecture

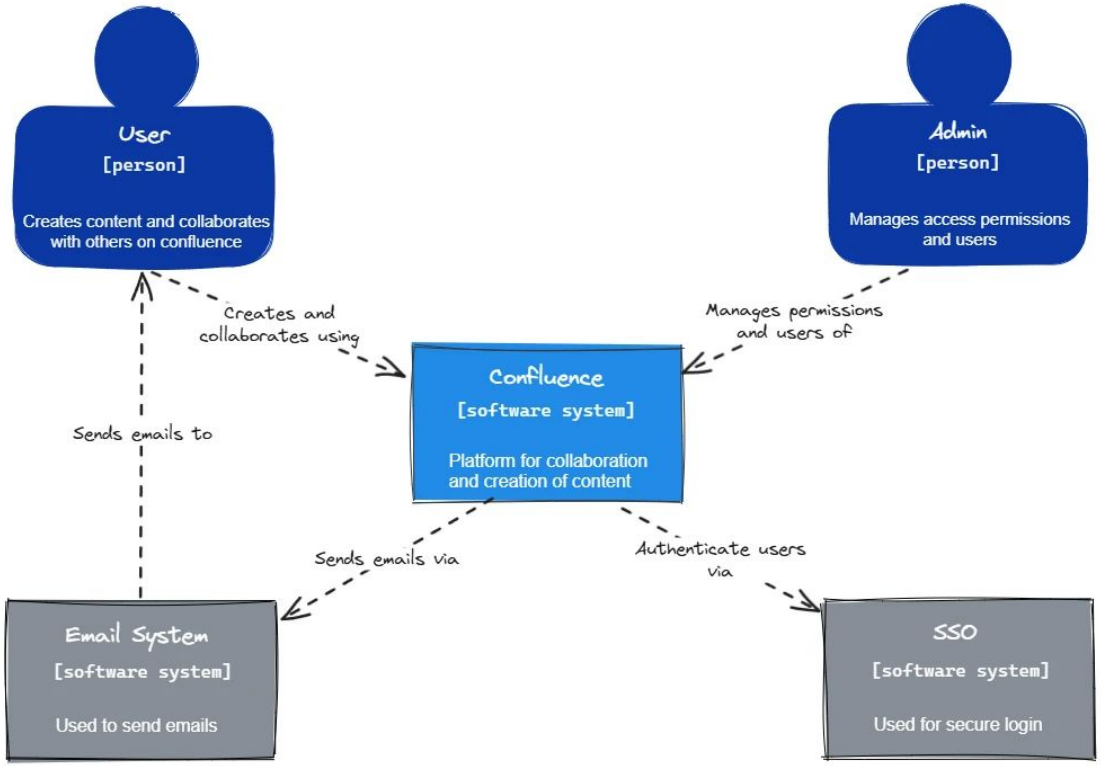
- Microservices are single-purpose, individually deployed units.
- Each microservice manages its own data and performs one specific function.
- Microservice architectures organise systems around these microservices and the communication between different microservices.



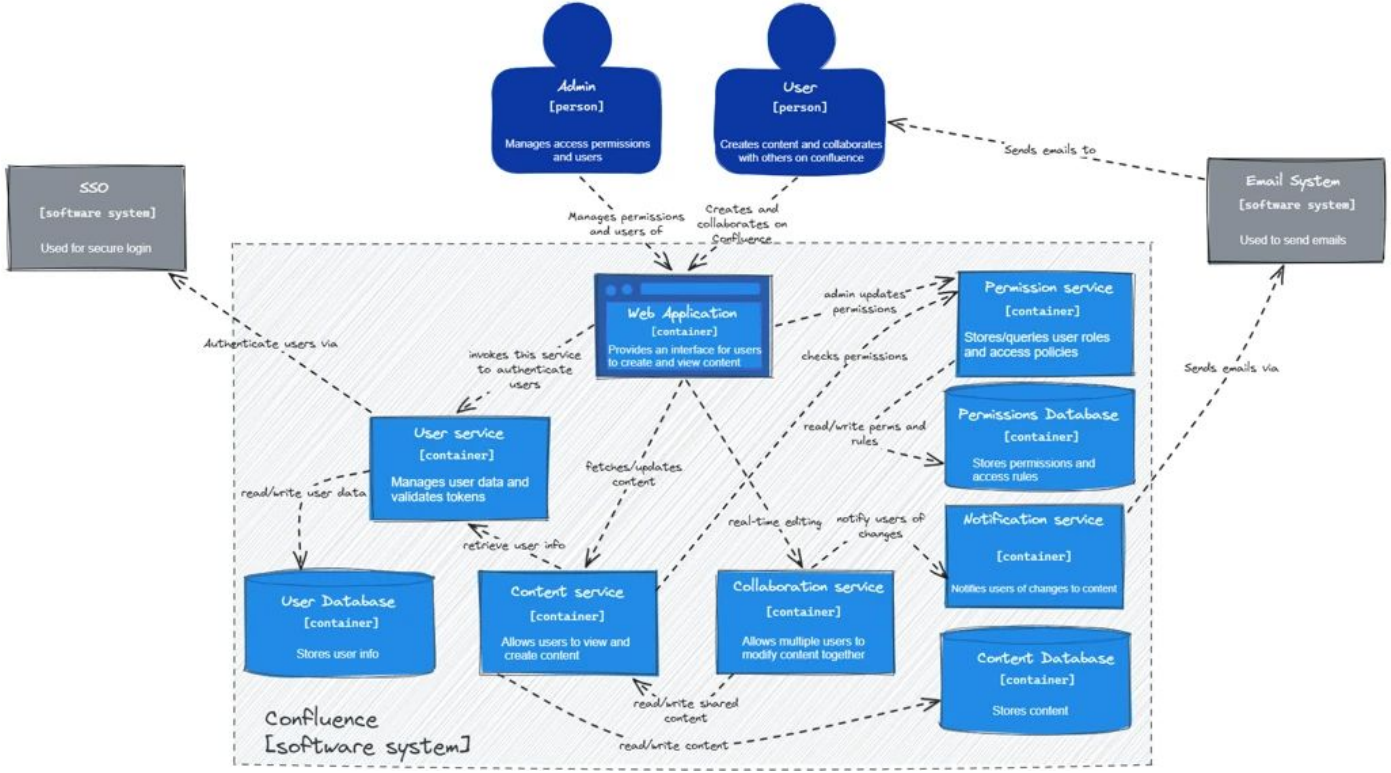
Microservice Architecture

- Relation to architectural characteristics:
  - **Scalability**: each microservice can be scaled independently.
  - **Fault Tolerance**: a fault in one microservice does not mean the entire system will break down, as each microservice is deployed independently.
  - **Maintainability**: changes are isolated to individual microservices, such as bug fixes.
  - **Security**: more 'access points' for attacks ('higher attack surface') as communication between microservices can be a vulnerability, but one microservice being compromised (likely) will not compromise another.
  - **Complexity** and **Cost**: a lot of infrastructure and networking is required to set up a distributed system compared to a monolith.
  - **Data Consistency**: data within the same microservice is fine (each microservice manages its own data), but needs to be synchronised across different microservices.

Microservices Context Diagram (Confluence)



Microservices Container Diagram (Confluence)

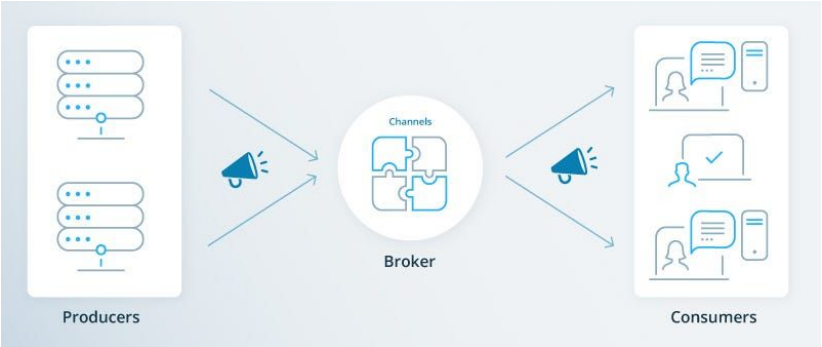


Event-Driven Architecture

- Event-driven architectures structure systems around components that broadcast or respond to *events*, which are significant changes in the system state (somewhat similar to the Observer Pattern).
- Events are communicated *asynchronously*; system components send messages without waiting for responses.
- An event-driven architecture can be applied to a monolithic or distributed system, but is most commonly associated with distributed systems.

Event-Driven Architecture

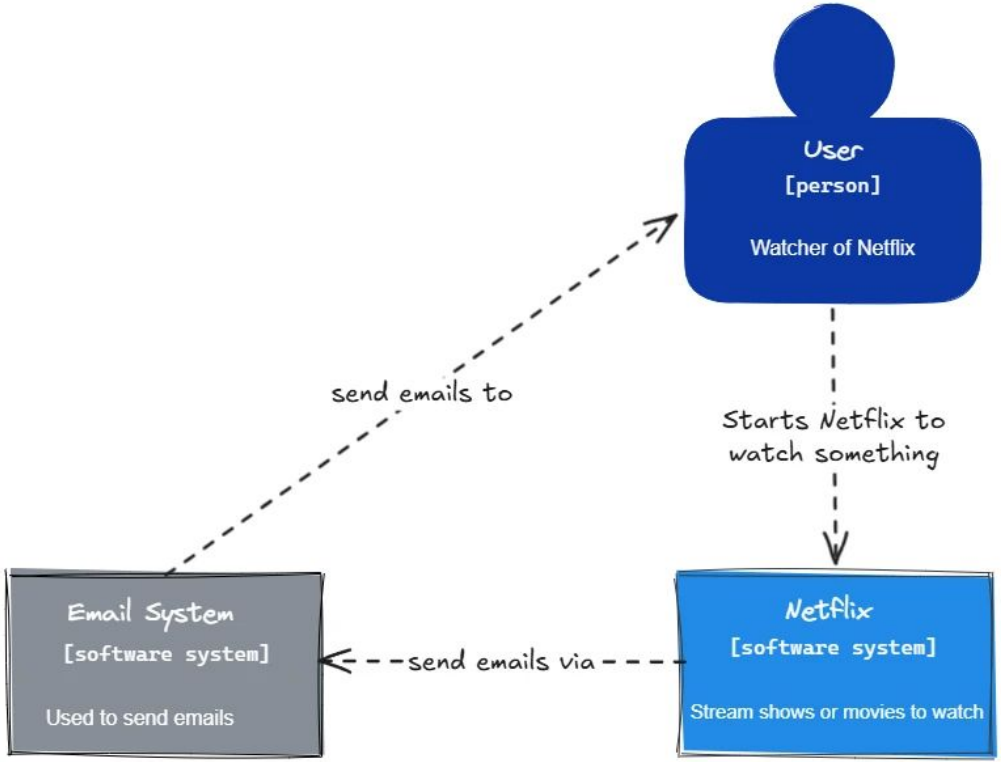
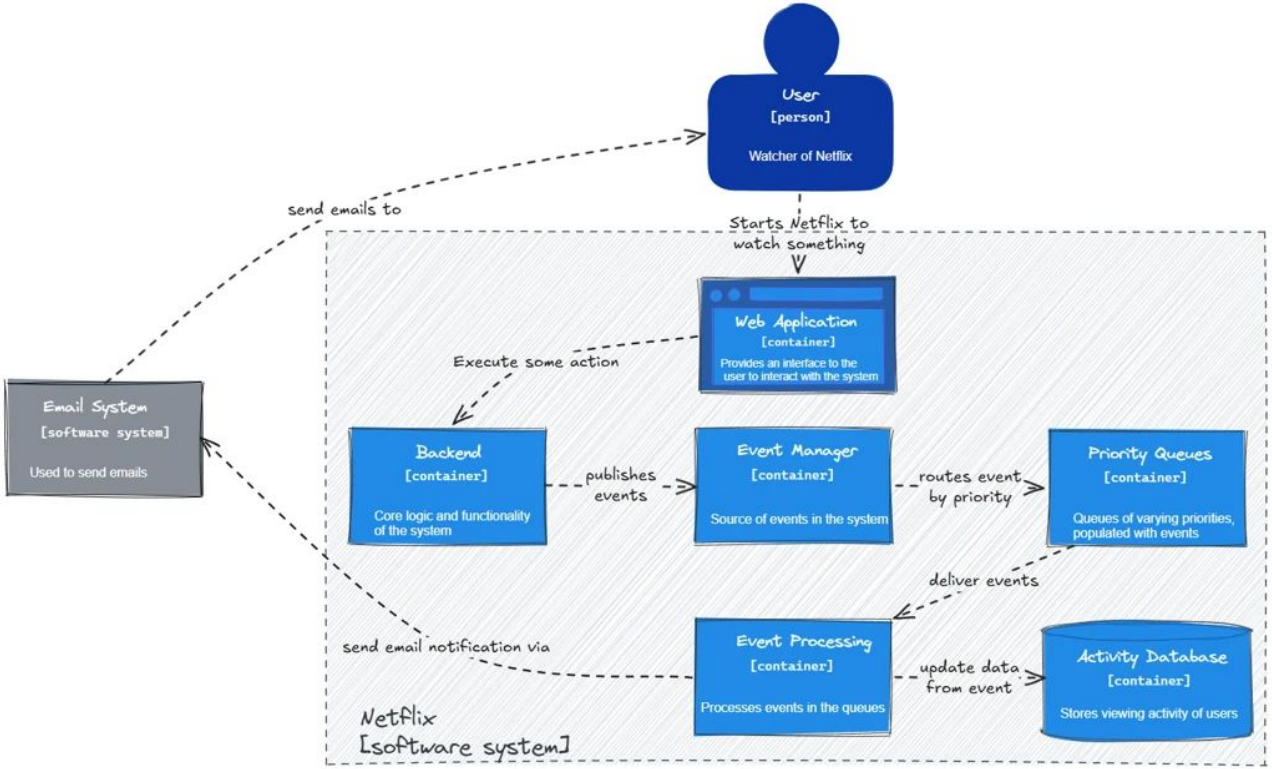
- An event-driven architecture typically uses a **broker**, which acts as a middle-man between publishers and consumers (subjects/observers).
- Events (e.g. user actions, database updates) are detected by **publishers**, which are sent to the event broker.
- The broker then relays them to relevant **consumers**, which react to the event.
- The use of a broker decouples publishers and consumers, serves as storage for events, allows messages to be ordered (if required) and can act as buffers.



Event-Driven Architecture

- Relation to architectural characteristics:
  - **Responsiveness**: real-time events can propagate throughout the system very quickly.
  - **Scalability**: easy to scale, since more events → more publishers and subscribers.
  - **Decoupling**: the use of a broker decouples publishers and subscribers.
  - **Performance**: asynchronous messaging allows publishers to send messages and move on without having to wait on a response, but practically there may be latency involved.
  - **Complexity**: asynchronous messages can be difficult to understand and reason with, since actions don't necessarily occur in pre-defined orders. Also, hard to set up.
  - **Testability**: testing and debugging asynchronous code is also complicated, as behaviour is non-deterministic and issues are hard to isolate.
  - **Data Consistency**: if applied to a distributed system, has eventual consistency.

EDA Context Diagram (Netflix)



EDA Container Diagram (Netflix)